RAG-Based Customer Support Assistant (LangGraph + HITL)

🟢 1. Introduction

🔹 What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that improves LLM responses by combining:

- **Retrieval** of relevant information from a knowledge base
- **Generation** of answers using an LLM

Instead of relying only on model knowledge, RAG ensures answers are **context-aware and accurate**.

🔹 Why RAG is Needed

- Reduces hallucination
- Improves accuracy
- Uses real-time document knowledge
- Enables domain-specific responses

🔹 Use Case

This project implements a **Customer Support Assistant** that:

- Answers user queries from a PDF knowledge base
- Escalates complex queries using **Human-in-the-Loop (HITL)**

🟢 2. System Architecture Explanation

The system consists of the following components:

🔹 Document Ingestion Pipeline

- Loads PDF using loader
- Splits into chunks
- Converts into embeddings

🔹 Vector Database (ChromaDB)

- Stores embeddings
- Performs similarity search

🔹 **Retrieval Layer**

- Retrieves relevant chunks based on query
- Uses semantic similarity

🔹 **LLM Processing Layer**

- Takes query + retrieved context
- Generates final answer

🔹 **Workflow Orchestration (LangGraph)**

- Controls system flow
- Routes decisions

🔹 **HITL Module**

- Handles complex queries
- Escalates to human

🟢 **3. Design Decisions**

🔹 **Chunk Size**

- Moderate chunk size used
- Ensures:
    - Better context
    - Faster retrieval

🔹 **Embedding Strategy**

- HuggingFace embeddings used
- Converts text into vectors

🔹 **Retrieval Approach**

- Top-K similarity search
- Retrieves most relevant chunks

🔹 **Prompt Design**

- Context + query passed to LLM
- Ensures grounded answers

🟢 **4. Workflow Explanation**

🔹 **LangGraph Workflow**

Nodes:

- **Process Node** → runs RAG pipeline
- **Decision Node** → checks confidence
- **HITL Node** → escalates

🔹 **State Management**

State contains:

- Query
- Answer
- Confidence
- Status

🔹 **Flow**

Input → Process → Decision → Output / HITL

🟢 **5. Conditional Logic**

🔹 **Decision Criteria**

- If confidence ≥ threshold → return answer
- If confidence < threshold → HITL

🔹 **Other Conditions**

- Sensitive queries (fraud/legal) → HITL
- Missing context → HITL

🟢 6. HITL Implementation

🔹 **When Triggered**

- Low confidence
- Sensitive queries
- No relevant data

🔹 **What Happens**

- Query is passed to human agent
- System returns escalation message

🔹 **Integration**

- HITL node in LangGraph
- Overrides automated response

🟢 7. Challenges & Trade-offs

🔹 **Challenges**

- Selecting optimal chunk size
- Improving retrieval accuracy
- Managing workflow state

🔹 **Trade-offs**

Factor	Trade-off
Chunk Size	Context vs Speed
Retrieval	Accuracy vs Latency
LLM Usage	Cost vs Performance

🟢 8. Testing Strategy

🔹 Testing Approach

- Functional testing
- Query-based testing

🔹 Sample Queries

- What is refund policy?
- How to cancel order?
- I want to report fraud

🔹 Expected Results

- Accurate answers for normal queries
- HITL trigger for complex queries

🟢 9. Future Enhancements

- Multi-document support
- Chat memory
- Web UI (Streamlit)
- Feedback loop
- Deployment

🟢 10. Conclusion

This project demonstrates how:

- RAG improves accuracy
- LangGraph enables decision-making
- HITL ensures reliability

The system is scalable, modular, and suitable for real-world applications.